23
PRESENTERS AND HUMBLE OBJECTS

In Chapter 22, we introduced the notion of presenters. Presenters are a form of the *Humble Object* pattern, which helps us identify and protect architectural boundaries. Actually, the Clean Architecture in the last chapter was full of *Humble Object* implementations.

THE HUMBLE OBJECT PATTERN

The *Humble Object* pattern[1] is a design pattern that was originally identified as a way to help unit testers to separate behaviors that are hard to test from behaviors that are easy to test. The idea is very simple: Split the behaviors into two modules or classes. One of those modules is humble; it contains all the hard-to-test behaviors stripped down to their barest essence. The other module contains all the testable

behaviors that were stripped out of the humble object.

For example, GUIs are hard to unit test because it is very difficult to write tests that can see the screen and check that the appropriate elements are displayed there. However, most of the behavior of a GUI is, in fact, easy to test. Using the *Humble Object* pattern, we can separate these two kinds of behaviors into two different classes called the Presenter and the View.

PRESENTERS AND VIEWS

The View is the humble object that is hard to test. The code in this object is kept as simple as possible. It moves data into the GUI but does not process that data.

The Presenter is the testable object. Its job is to accept data from the application and format it for presentation so that the View can simply move it to the screen. For example, if the application wants a date displayed in a field, it will hand the Presenter a `Date` object. The Presenter will then format that data into an appropriate string and place it in a simple data structure called the View Model, where the View can find it.

If the application wants to display money on the screen, it might pass a `Currency` object to the Presenter. The Presenter will format that object with the appropriate decimal places and currency markers, creating a string that it can place in the View Model. If that currency value should be turned red if it is negative, then a simple boolean flag in the View model will be set appropriately.

Every button on the screen will have a name. That name will be a string in the View Model, placed there by the presenter. If those buttons should be grayed out, the Presenter will set an appropriate boolean flag in the View model. Every menu item name is a string in the View model, loaded by the Presenter. The names for every radio button, check box, and text field are loaded, by the Presenter, into appropriate strings and booleans in the View model. Tables of numbers that should be displayed on the screen are loaded, by the Presenter, into tables of properly formatted strings in the View model.

Anything and everything that appears on the screen, and that the application has some kind of control over, is represented in the View Model as a string, or a boolean, or an enum. Nothing is left for the View to do other than to load the data from the View Model into the screen. Thus the View is humble.

TESTING AND ARCHITECTURE

It has long been known that testability is an attribute of good architectures. The *Humble Object* pattern is a good example, because the separation of the behaviors into testable and non-testable parts often defines an architectural boundary. The Presenter/View boundary is one of these boundaries, but there are many others.

DATABASE GATEWAYS

Between the use case interactors and the database are the database gateways.[2] These gateways are polymorphic interfaces that contain methods for every create, read, update, or delete operation that can be performed by the application on the database. For example, if the application needs to know the last names of all the users who logged in yesterday, then the `UserGateway` interface will have a method named `getLastNamesOfUsersWhoLoggedInAfter` that takes a `Date` as its argument and returns a list of last names.

Recall that we do not allow SQL in the use cases layer; instead, we use gateway interfaces that have appropriate methods. Those gateways are implemented by classes in the database layer. That implementation is the humble object. It simply uses SQL, or whatever the interface to the database is, to access the data required by each of the methods. The interactors, in contrast, are not humble because they encapsulate application-specific business rules. Although they are not humble, those interactors are *testable*, because the gateways can be replaced with appropriate stubs and test-doubles.

DATA MAPPERS

Going back to the topic of databases, in which layer do you think ORMs like Hibernate belong?

First, let’s get something straight: There is no such thing as an object relational mapper (ORM). The reason is simple: Objects are not data structures. At least, they are not data structures from their users’ point of view. The users of an object cannot see the data, since it is all private. Those users see only the public methods of that object. So, from the user’s point of view, an object is simply a set of operations.

A data structure, in contrast, is a set of public data variables that have no implied behavior. ORMs would be better named “data mappers,” because they load data into data structures from relational database tables.

Where should such ORM systems reside? In the database layer of course. Indeed, ORMs form another kind of *Humble Object* boundary between the gateway interfaces and the database.

SERVICE LISTENERS

What about services? If your application must communicate with other services, or if your application provides a set of services, will we find the *Humble Object* pattern creating a service boundary?

Of course! The application will load data into simple data structures and then pass those structures across the boundary to modules that properly format the data and send it to external services. On the input side, the service listeners will receive data from the service interface and format it into a simple data structure that can be used by the application. That data structure is then passed across the service boundary.

CONCLUSION

At each architectural boundary, we are likely to find the *Humble Object* pattern lurking somewhere nearby. The communication across that boundary will almost always involve some kind of simple data structure, and the boundary will frequently divide something that is hard to test from something that is easy to test. The use of this pattern at architectural boundaries vastly increases the testability of the entire system.

1. *xUnit Patterns*, Meszaros, Addison-Wesley, 2007, p. 695.
2. *Patterns of Enterprise Application Architecture*, Martin Fowler, et. al., Addison-Wesley, 2003, p. 466.